



SLEEP STAGES CLASSIFICATION

Using deep learning

Group 3: Mennat Allah Mohamed Elsayed

Sohaila Elsayed Ibrahim

Wafaa Hassan Sharaan

Under Supervision of: Dr. Ibrahim Sadek

Eng. Veronica William

Biomedical Engineering Year (3) – 2nd Semester

2025 - 2026

INTRODUCTION:

Sleep is a complex physiological process with cyclical stages such as neural activity, eye movement behavior, muscular activity, and autonomic regulation. Accurate sleep staging is important for the diagnosis of sleep problems such as insomnia, obstructive sleep apnea, narcolepsy, REM behavior disorder, and circadian rhythm problems.

Normal sleep rating is based on manual interpretation of polysomnography recordings by sleep technicians according to AASM standards. Manual rating is accurate but requires so much clinical effort and is not practical for large scale continuous monitoring applications.

Latest developments in deep learning have made up the possibility of directly extracting discriminative features from biomedical signals without manual feature engineering. In this project, EOG signals were selected as eye movement patterns had a clear connection with the sleep stages, especially REM transitions and waking state.

In this project, we analyze the use of deep neural networks to classify sleep stages directly from the EOG epochs. The architecture was slowly improved through several experimental versions.

PROBLEM STATEMENT:

- Manual sleep stage scoring is exhausting, expensive and extremely dependent on trained sleep experts. The accuracy of sleep stage annotation may also be affected by inter scorer variability.
- Traditional polysomnography (PSG) requires multiple physiological signals, which increase the complexity of the system and limit the use of portable or personal monitoring.
- Automatic sleep staging with EOG signals faces challenges such as signal noise, class imbalance between sleep stages and the need for robust feature extraction from 1D time series data.
- so, this work aims to develop an efficient deep learning model for automatic sleep stage classification based only on EOG signals with high accuracy and low computational complexity.

OBJECTIVES:

- Design an automatic end to end deep learning model for sleep stage classification using EOG signals.
- Design and evaluate a hybrid 1D-CNN and BiLSTM model to learn spatial features and temporal dependencies.
- Reduce dataset imbalance through specific augmentation and sampling strategies.
- Slowly improve the model architecture and compare intermediate models to a final, high performing model.

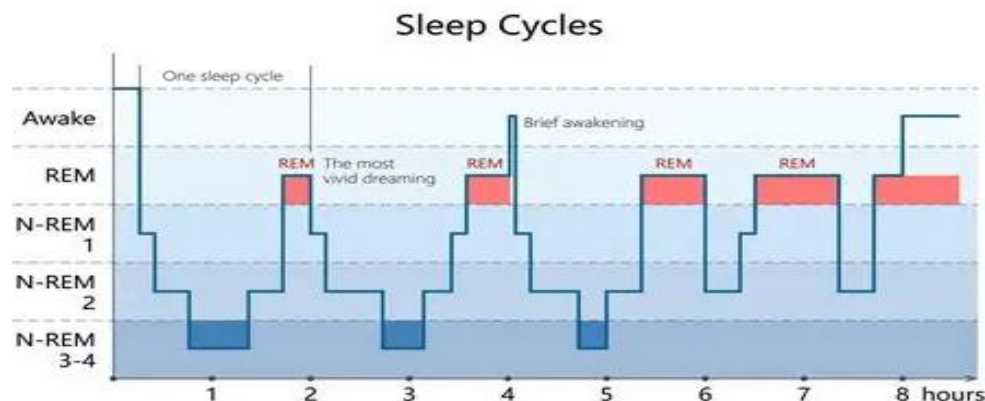
BACKGROUND:

1. Sleep staging is commonly divided into five classes:

- Wake stage (High activity of eye movement and irregular behavior.)
- N1 stage (means light sleep with slow rolling eye movements.)
- N2 stage (Intermediate sleep stage with reduced ocular activity.
- N3 stage (Deep sleep stage with slow wave activity.)
- REM stage (defined as rapid eye movements and vivid dreaming activity.)

The classification task is to classify each 30-sec EOG epoch into one of these stages.

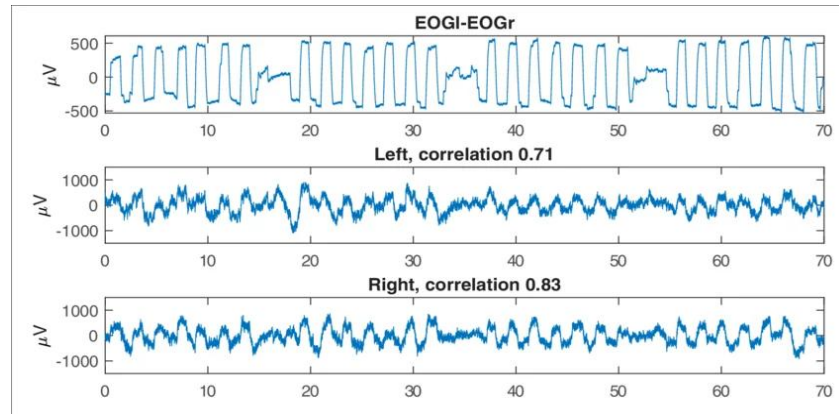
Each stage has its own physiological characteristics. EOG signals contain eye movement dynamics that are very useful for transitions between Wake, N1 and REM stages.



2. EOG Signal Explanation

Electrooculography (EOG) measures the resting corneal retinal potential, which is sensitive to eye movements. Quick eye movements are detected in the EOG signal during the Wake stage and REM sleep, which causes frequent, large amplitude deflections. However, N1 is characterized by slow eye movements (SEM) while N2,N3 shows

relatively little eye activity. The use of 1D convolutional neural networks on these signals allows us to extract morphological patterns associated with specific sleep stages without large EEG arrays.



Dataset Description:

MESA (Multi-Ethnic Study of Atherosclerosis) Sleep Dataset.

Subjects: Data from 10 individuals were used (IDs: 0001, 0002, 0006, 0010, 0012, 0014, 0016, 0021, 0027, 0028).

Format: Data consists of

- **.edf** files consist of time-series physiological signals. These signals provide valuable information about brain activity and physiological changes occurring during different sleep stages.

- **XML**

file acts as a "Time-Stamped Log" or "Metadata Map" created by sleep technicians to describe the continuous signals found in the EDF file sleep Staging

PROJECT PIPELINE:

All models we made follow the same preprocessing pipeline

1. Data Reading:

Check if the path contains data folders or not and print file names.

Check signal channels, print the head of one patient's signal file .

2. Annotations labeling:

from XML files, epoching the annotations into 30 second epochs, specifying the stage in each epoch and giving every stage a label .

STAGES	LABLES
Wake stage	0
N1 stage	1
N2 stage	2
N3 stage	3
REM stage	4

Then save it in CSV files

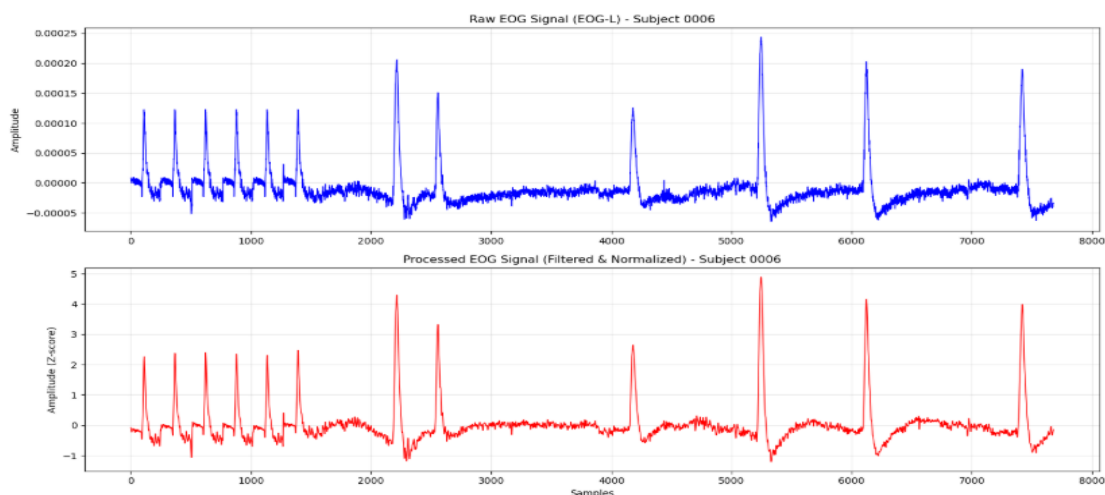
3. Preprocessing:

- Channel selection** (left eog channel)
- Bandpass filter** (0.3-30 HZ) is applied to the EOG signals to remove unwanted frequencies
- Z-score normalization** to improve convergence & stabilizing the training
- Epoching** into 30 seconds to match and align the annotations, given a sampling rate of 256 Hz, each epoch contains 30*256 data points

4. Visualizing the difference between raw& processed data:

print the following values: mean, std, min, max, and range of both from one patient and graph the duration of signals

As shown in the attached figures:



```

Raw Signal Length: 8294144 samples
Processed File Loaded - Number of epochs: 1079

--- Raw vs Processed - Subject 0006 (First Epoch) ---
Metric      Raw Signal      Processed Signal      Diff
Mean        -0.000009      -0.002399      -0.002390
Std         0.000035      0.667366      0.667331
Min         -0.000065      -1.204099      -1.204034
Max         0.000244      4.882393      4.882149
Range       0.000309      6.086492

First 15 values:
Sample 0 | Raw: 0.00001 | Processed: -0.08014 | Diff: -0.08014
Sample 1 | Raw: 0.00001 | Processed: -0.09835 | Diff: -0.09835
Sample 2 | Raw: 0.00001 | Processed: -0.11717 | Diff: -0.11717
Sample 3 | Raw: 0.00000 | Processed: -0.13701 | Diff: -0.13702
Sample 4 | Raw: 0.00000 | Processed: -0.15782 | Diff: -0.15782
Sample 5 | Raw: 0.00000 | Processed: -0.17868 | Diff: -0.17869
Sample 6 | Raw: 0.00000 | Processed: -0.19773 | Diff: -0.19774
Sample 7 | Raw: 0.00000 | Processed: -0.21251 | Diff: -0.21251
Sample 8 | Raw: -0.00000 | Processed: -0.22084 | Diff: -0.22084
Sample 9 | Raw: 0.00000 | Processed: -0.22174 | Diff: -0.22174
Sample 10 | Raw: 0.00000 | Processed: -0.21554 | Diff: -0.21554
Sample 11 | Raw: -0.00000 | Processed: -0.20353 | Diff: -0.20353
Sample 12 | Raw: 0.00000 | Processed: -0.18751 | Diff: -0.18751
Sample 13 | Raw: 0.00001 | Processed: -0.16951 | Diff: -0.16952
Sample 14 | Raw: 0.00000 | Processed: -0.15166 | Diff: -0.15166

```

Preprocessed data vs Raw data

5. Data splitting:

We applied two methods of data splitting:

1. Sampling (70% for training - 15 % for validation - 15% for testing)
2. Subject wise splitting (7 for training - 1 for validation - 2 for testing)

To see the difference.

MODEL ARCHITECTURE:

The proposed model, named SleepClass. It integrates convolutional neural networks (CNNs) with recurrent neural networks (RNNs) and an attention mechanism to effectively capture both local and long-range dependencies within physiological signals. The architecture is composed of three main custom-defined modules: an Attention layer, a ResidualBlock, and the overarching SleepClass model.

Attention Layer:

The Attention layer (`Attention(nn.Module)`) is a crucial component that allows the model to focus on the most relevant parts of the input sequence. It takes a hidden state representation (e.g., from an LSTM) and computes a weighted sum of its features. The weights are learned through a linear transformation followed by a softmax activation function, ensuring that the sum of weights for each input sequence sums to one. This mechanism enhances the model's ability to highlight salient features for classification.

Residual Block:

The `ResidualBlock` (`ResidualBlock(nn.Module)`) is a building block inspired by ResNet architectures, designed to mitigate the vanishing gradient problem and facilitate the training of deeper networks. Each block consists of a main path with a 1D convolutional layer, batch normalization, and a ReLU activation function. A shortcut connection directly adds the input to the output of the main path. If the input and output dimensions differ (due to stride or channel changes), a 1x1 convolutional layer with batch normalization is applied to the shortcut path to match the dimensions. This design allows for the learning of identity functions, improving information flow through the network.

SleepClass Model:

The SleepClass model combines these components to process sequential input data, such as physiological signals for sleep staging. The model's forward pass can be summarized as follows:

1. Feature Extraction with Residual Blocks and Pooling: The input signal first passes through a series of three `ResidualBlock` layers, each followed by a `MaxPool1d` layer. This hierarchical processing extracts increasingly

abstract features while progressively reducing the spatial dimensions of the input

2. Permutation: The output from the convolutional layers is then permuted to match the expected input format of the LSTM layer (batch_size, sequence_length, features).

3. Bidirectional LSTM: A bidirectional Long Short-Term Memory (LSTM) layer processes the sequential features. The bidirectional nature allows the model to capture dependencies from both past and future contexts, which is particularly beneficial for time-series data. The LSTM is configured with an input size of 128, a hidden size of 64, and batch_first=True.

4. Attention Mechanism: The output of the LSTM layer is then fed into the custom Attention layer. This step allows the model to weigh the importance of different time steps in the sequence, producing a context vector that emphasizes the most relevant information for the final classification.

5. Classifier: Finally, a fully connected (FC) layer (nn.Sequential) acts as the classifier. It consists of two linear layers separated by a ReLU activation and a Dropout layer (with a dropout rate of 0.5) to prevent overfitting. The final linear layer outputs num_classes (defaulting to 5), corresponding to the different sleep stages.

TRAINING AND EVALUATION:

We divided our EOG data into three data loaders (train, validation, and test) with a batch size of 32 to train our sleep staging model (SleepClass). The model was designed to classify five sleep stages.

We use AdamW optimizer with an initial learning rate of $1e-3$ and weight decay of $1e-3$ to optimize the model. Sleep data can be tricky and models tend to overfit, hence we applied a label smoothing of 0.1 to our Cross-Entropy loss function. This helps the model to not be too confident in its predictions and improves generalization.

Training

Training process was planned for 50 epochs. We didn't use a fixed learning rate in the training loop, but instead used a OneCycleLR scheduler. By this way, the learning rate could increase and decrease dynamically, which led to faster model convergence and avoided local minima. We also applied gradient clipping with a max norm of 1.0 to maintain stability during backpropagation and prevent exploding gradients.

We also recorded loss and accuracy per epoch to monitor the model's learning process.

Model Checkpointing & Validation

In each epoch, we updated the weights during training phase and then evaluated the model on validation set. We also monitored the validation loss to ensure we didn't end up with an overfitted model at the end of the 50 epochs. We copied over when the model had a lower validation loss than the previous best. `deepcopy` to save weights (`best_sleep_model.pth`). This checkpointing scheme made sure we kept the best version of our model. We also stored the loss and accuracy metrics for all epochs, to later visualize the learning curves.

Final evaluation

Once the training phase was fully completed, we proceeded to the final evaluation using our unseen test dataset. We loaded the weights of the best saved model and set it to evaluation mode. We passed the test data through the model and applied softmax function to get the prediction probabilities. Finally, we compared the predicted classes of the model with the actual ground truth labels to obtain the final test accuracy

INDIVIDUAL PROJECT ANALYSIS AND DIFFERENCE:

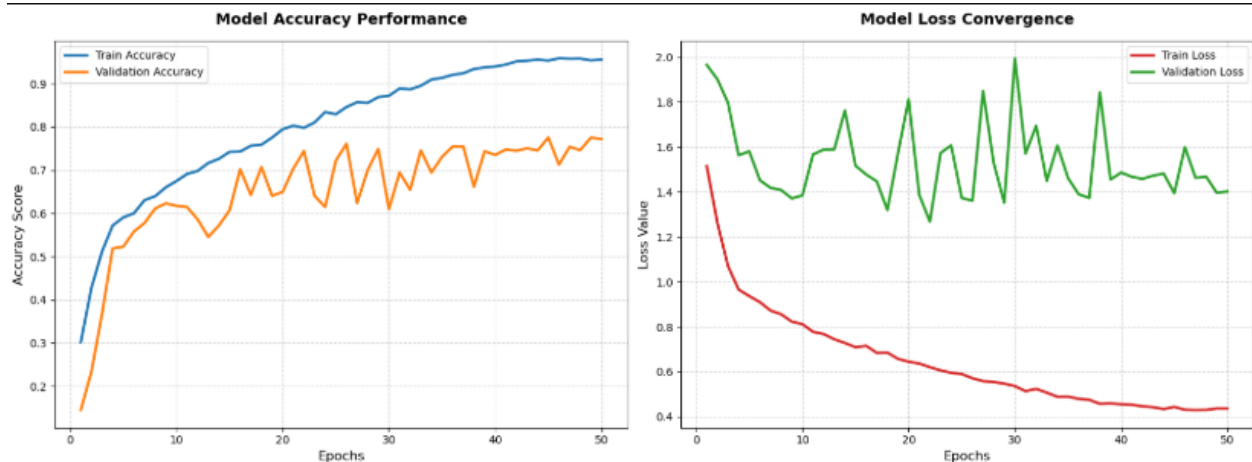
1. [Signals eog1.ipynb](#)

- Data Splitting: subject-wise splitting (7 subjects for training, 1 for validation, 2 for testing).
- Data Imbalance Handling: Uses WeightedRandomSampler in DataLoader during training. The technique gives higher weights to the samples of under-represented classes, increasing the chance of those samples being selected for training and helping the model to learn better from those classes.
- Data Augmentation: Uses simple data augmentation techniques such as random noise addition and signal scaling. These techniques help to make the model robust and generalizable.
- Loss Function: CrossEntropyLoss with class weights, calculated to balance the data imbalance. This makes misclassification of underrepresented classes contribute more

to the loss, forcing the model to improve its performance on those classes.

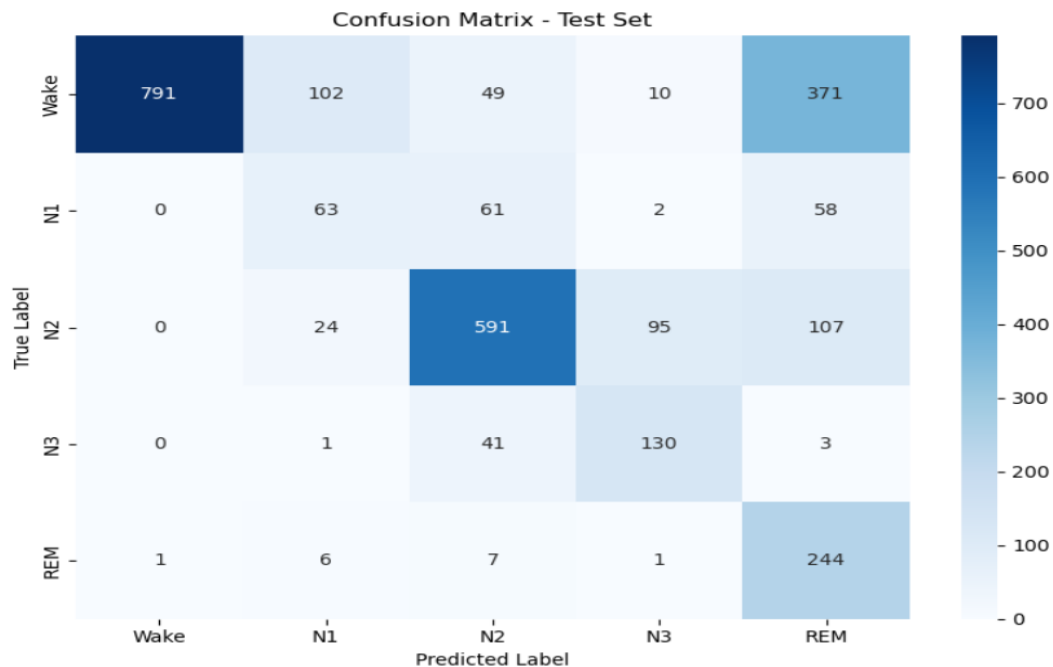
```
Epoch 49/50 [Train]: 100% | 267/267 [00:24<00:00, 10.95it/s, Loss=0.479, LR=2e-6]
Train Loss: 0.4366 | Train Acc: 0.9542 | Val Loss: 1.3961 | Val Acc: 0.7751
Epoch 50/50 [Train]: 100% | 267/267 [00:24<00:00, 11.04it/s, Loss=0.408, LR=4.03e-9]
Train Loss: 0.4363 | Train Acc: 0.9560 | Val Loss: 1.4021 | Val Acc: 0.7722
Training completed!
```

Training & validation accuracy of "signals eog1" notebook



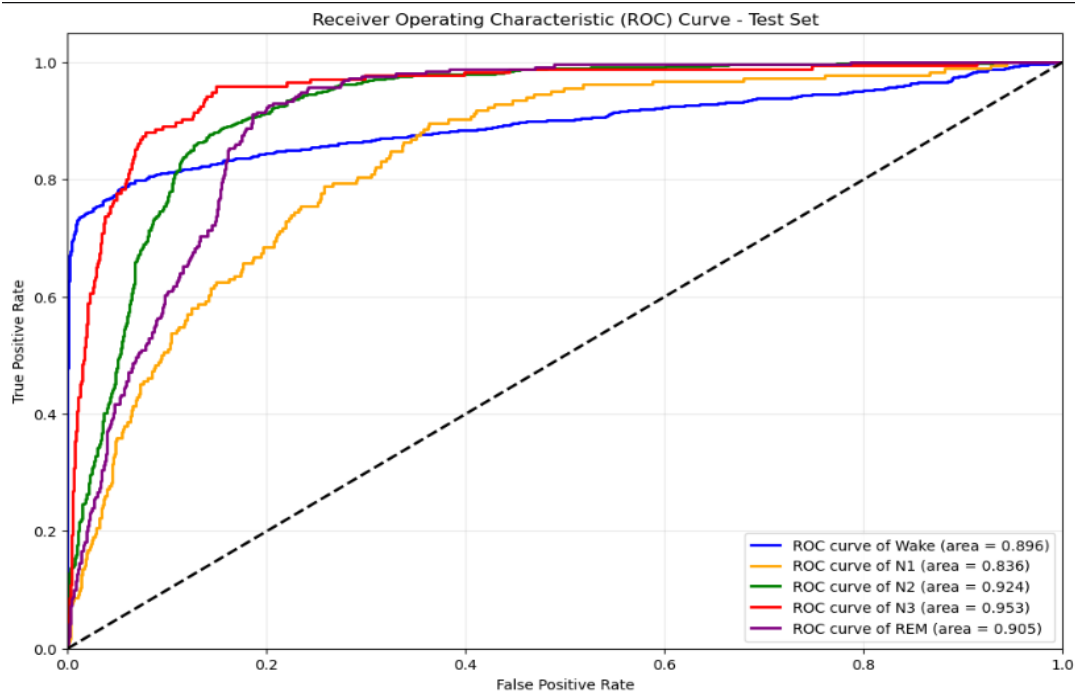
Final Test Accuracy: 65.95%

Test accuracy of "signals eog1" notebook



Classification Report:

	precision	recall	f1-score	support
Wake	0.9987	0.5979	0.7480	1323
N1	0.3214	0.3424	0.3316	184
N2	0.7891	0.7234	0.7548	817
N3	0.5462	0.7429	0.6295	175
REM	0.3116	0.9421	0.4683	259
accuracy			0.6595	2758
macro avg	0.5934	0.6697	0.5864	2758
weighted avg	0.7982	0.6595	0.6884	2758



2. [signalseog2.ipynb](#)

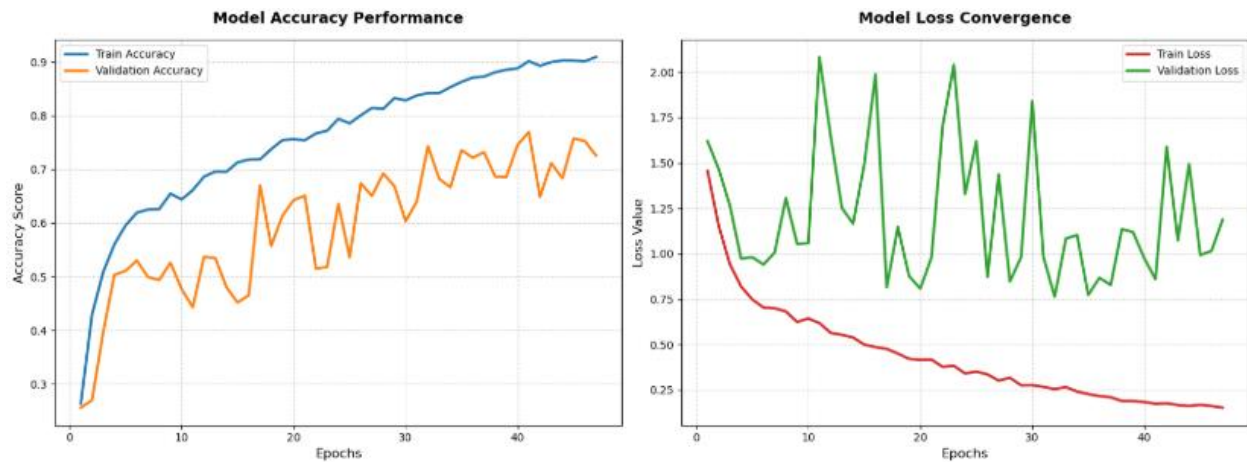
- Data splitting : sampling (70-15-15)
- Handling of data imbalance: Same as the first notebook, WeightedRandomSampler is used.
- Data Augmentation: In addition to the random noise, it also adds Time Shifting augmentation. This technique randomly shifts the

signal in time which allows the model to be more robust to variations in the timing of events in the physiological signal.

- Early stopping: An early stopping mechanism is used with patience of 15 epochs. This means that if the model performance on the validation set does not improve for 15 consecutive epochs, the training will stop to prevent overfitting and save computational resource.
- Loss Function: CrossEntropyLoss with class weight.

```
Epoch 9/50 [Train]: 100%|██████████| 279/279 [00:24<00:00, 11.45it/s, Loss=0.82, LR=0.000669]  
...  
Train Loss: 0.1521 | Train Acc: 0.9092 | Val Loss: 1.1881 | Val Acc: 0.7257  
No improvement. (Count: 15/15)  
Training stopped to prevent overfitting.  
Training completed!
```

Training & validation accuracy of "signals eog2" notebook



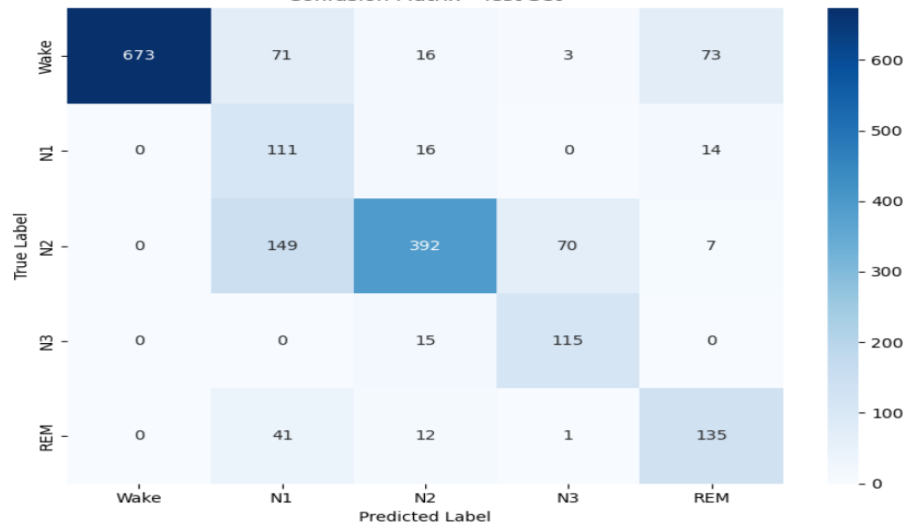
Final Test Accuracy: 74.50%

Test accuracy of "signals eog2" notebook

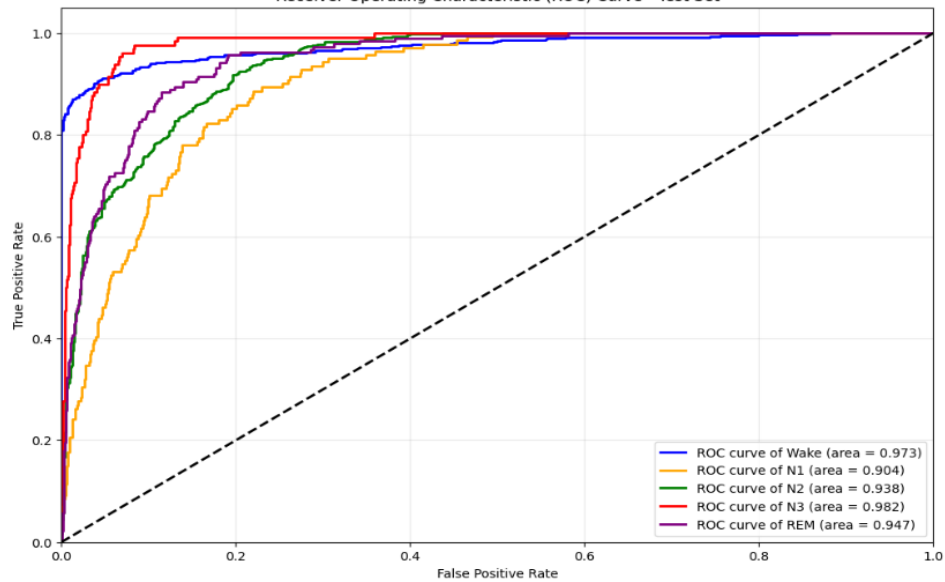
Classification Report:

	precision	recall	f1-score	support
Wake	1.0000	0.8050	0.8920	836
N1	0.2984	0.7872	0.4327	141
N2	0.8692	0.6343	0.7334	618
N3	0.6085	0.8846	0.7210	130
REM	0.5895	0.7143	0.6459	189
accuracy			0.7450	1914
macro avg	0.6731	0.7651	0.6850	1914
weighted avg	0.8389	0.7450	0.7710	1914

Confusion Matrix - Test Set



Receiver Operating Characteristic (ROC) Curve - Test Set

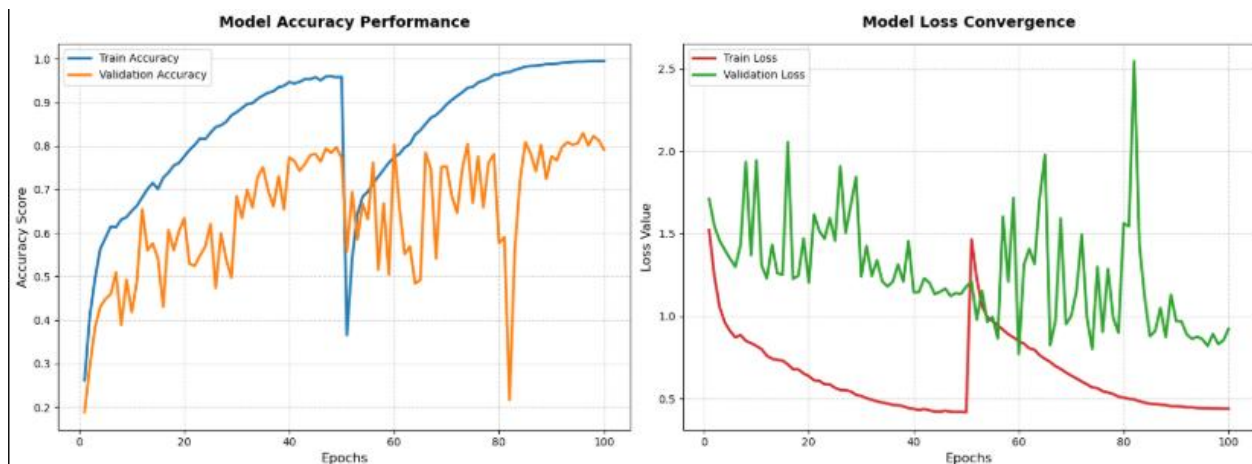


[3. signalseog3.ipynb](#)

- Data splitting : subject-wise
- Data Imbalance Handling: Applies SMOTE [Synthetic Minority Over-sampling Technique]. SMOTE creates new synthetic samples for minority classes, instead of just re-weighting samples. This effectively increases the amount of training data for rare classes, which helps the model learn decision boundaries more effectively.
- Data augmentation: The same random noise and scaling methods are used.
- Loss Function: The standard CrossEntropyLoss is used without explicit class weights as SMOTE has already taken care of the imbalance in data distribution.

```
Epoch 49/50 [Train]: 100%|██████████| 609/609 [00:51<00:00, 11.74it/s, Loss=0.435, LR=2.01e-6]
Train Loss: 0.4378 | Train Acc: 0.9949 | Val Loss: 0.8523 | Val Acc: 0.8130
Epoch 50/50 [Train]: 100%|██████████| 609/609 [00:53<00:00, 11.42it/s, Loss=0.436, LR=4.01e-9]
Train Loss: 0.4384 | Train Acc: 0.9948 | Val Loss: 0.9230 | Val Acc: 0.7915
Training completed!
```

Training & validation accuracy of "signals eog3" notebook



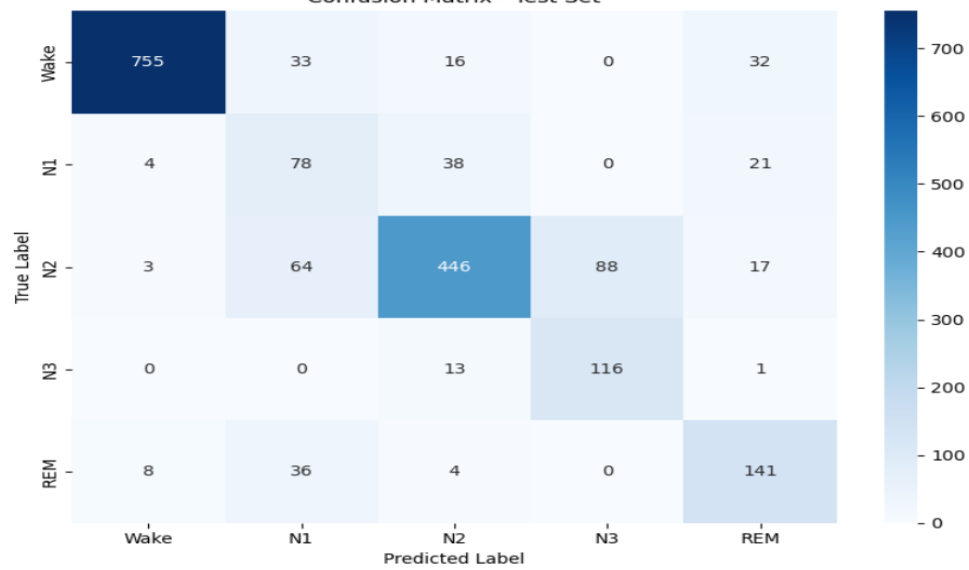
Final Test Accuracy: 80.25%

Test accuracy of "signals eog3" notebook

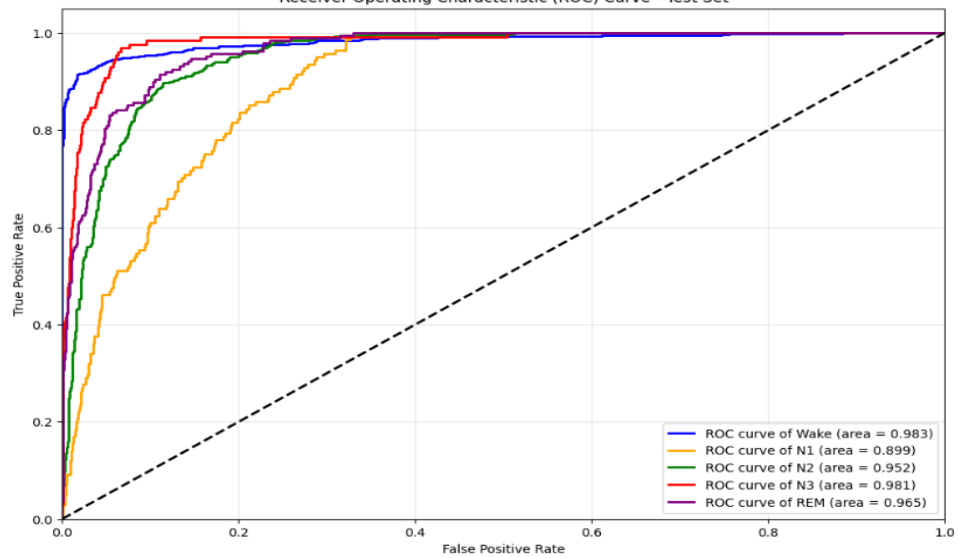
Classification Report:

	precision	recall	f1-score	support
Wake	0.9805	0.9031	0.9402	836
N1	0.3697	0.5532	0.4432	141
N2	0.8627	0.7217	0.7859	618
N3	0.5686	0.8923	0.6946	130
REM	0.6651	0.7460	0.7032	189
accuracy			0.8025	1914
macro avg	0.6893	0.7633	0.7134	1914
weighted avg	0.8383	0.8025	0.8137	1914

Confusion Matrix - Test Set



Receiver Operating Characteristic (ROC) Curve - Test Set



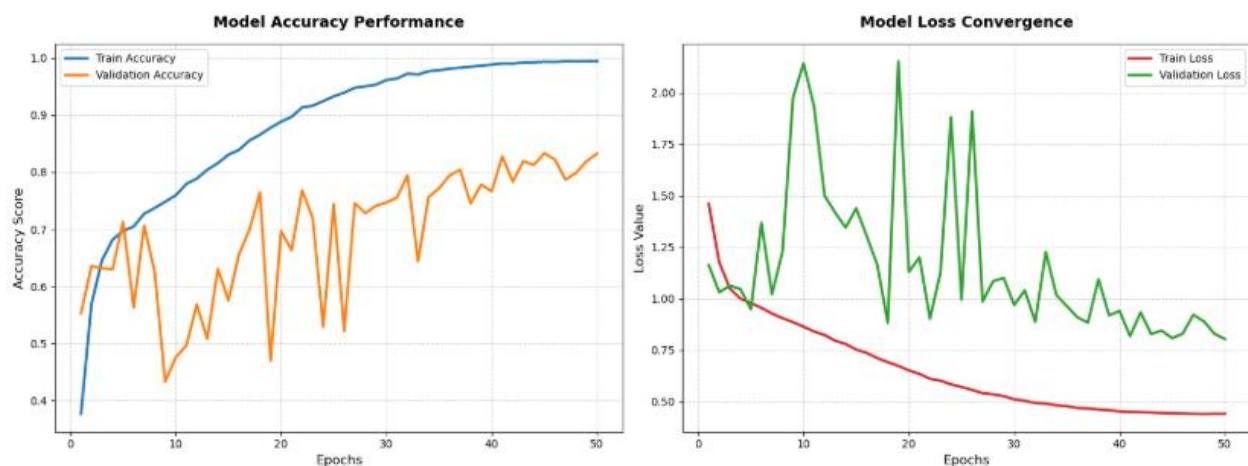
[4. signalseog4.ipynb](#) .

- Data splitting: sampling
- Data Imbalance Handling: Uses SMOTE in the same way as signalseog3.ipynb to boost the representation of minority classes.
- Data Augmentation: Uses the same technique of random noise and scaling.
- Loss Function: Uses CrossEntropyLoss with Label Smoothing at 0.1. Label Smoothing is used as a regularization technique which prevents the model from being over confident about its predictions.

Hard labels are replaced by a small share of the probability being assigned to all other classes. This reduces the overfitting of the model and improves the generalization ability of the model to new data.

```
Epoch 49/50 [Train]: 100%|██████████| 609/609 [00:58<00:00, 10.38it/s, Loss=0.42, LR=2.01e-6]  
Train Loss: 0.4410 | Train Acc: 0.9940 | Val Loss: 0.8303 | Val Acc: 0.8192  
Epoch 50/50 [Train]: 100%|██████████| 609/609 [01:02<00:00, 9.78it/s, Loss=0.426, LR=4.01e-9]  
Train Loss: 0.4406 | Train Acc: 0.9940 | Val Loss: 0.8041 | Val Acc: 0.8323  
Training completed!
```

Training & validation accuracy of "signals eog4" notebook

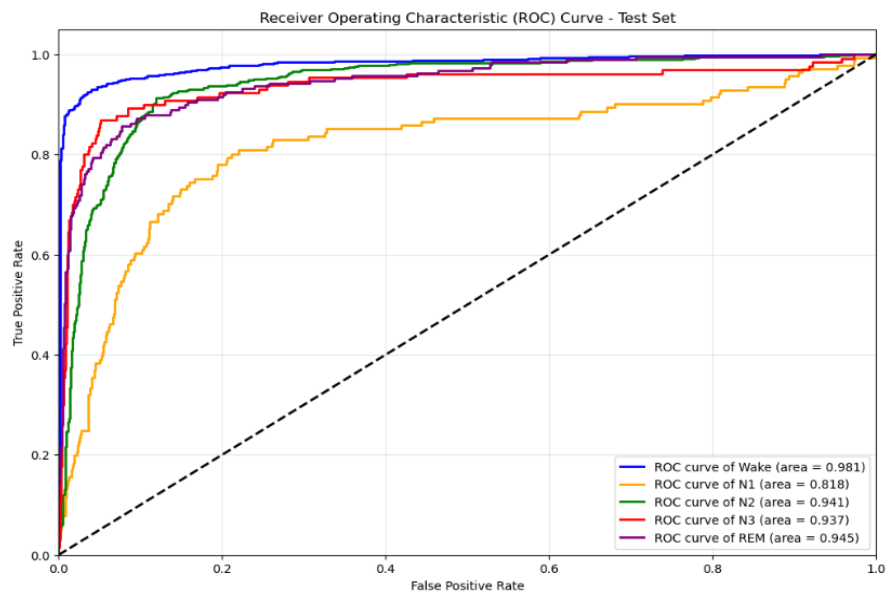
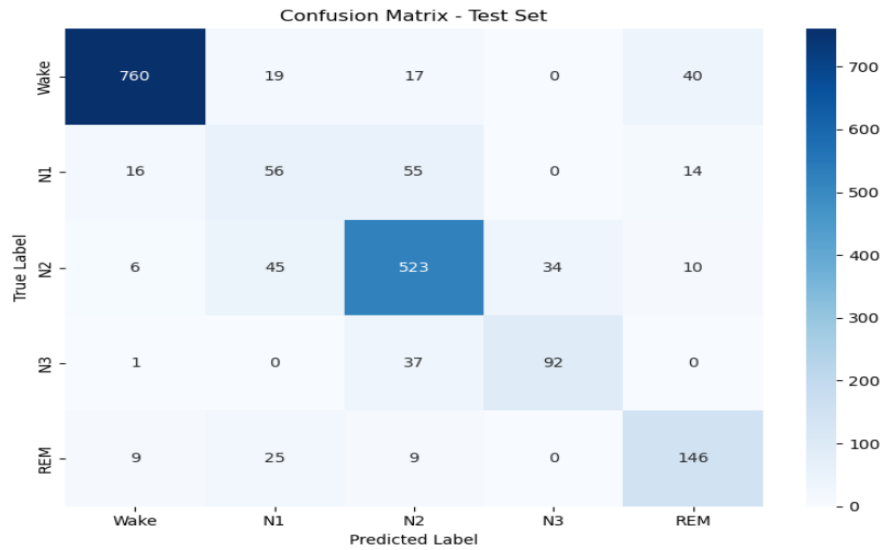


```
Final Test Accuracy: 82.39%
```

Test accuracy of "signals eog4" notebook

Classification Report:

	precision	recall	f1-score	support
Wake	0.9596	0.9091	0.9337	836
N1	0.3862	0.3972	0.3916	141
N2	0.8159	0.8463	0.8308	618
N3	0.7302	0.7077	0.7188	130
REM	0.6952	0.7725	0.7318	189
accuracy			0.8239	1914
macro avg	0.7174	0.7265	0.7213	1914
weighted avg	0.8293	0.8239	0.8260	1914



REFERENCE:

- The MESA (Multi-Ethnic Study of Atherosclerosis)
- [U-Sleep: resilient high-frequency sleep staging - PMC](#)